An Integrated E-Commerce Customer Behavior Analytics Platform Using the Olist Dataset

Sahil
Big Data Analytics Project, India

Abstract— E-commerce transaction logs contain signals about customer value, product affinity, geography, and sales dynamics, but these signals are difficult to use when data preparation, analytical code, and presentation are disconnected. This paper presents a local web-based analytics platform built with React, FastAPI, and PostgreSQL, adapted to the Brazilian E-Commerce Public Dataset by Olist. The system stores the nine relational Olist files in PostgreSQL, exposes compatibility views for a shared analytics model, and provides a user interface for sales dashboards, recency-frequency-monetary (RFM) analysis, K-Means customer segmentation, Apriori association rules, revenue forecasting, run history, and customer-to-customer recommendations. The recommender represents delivered purchases as product sets, ranks customers by Jaccard similarity, scores unseen products using neighbor similarity, and uses a popularity fallback. The contribution is an integrated analytics workflow rather than a new learning algorithm. Olist records describe historical Brazilian marketplace orders from 2016-2018; the application therefore does not establish current or India-specific consumer trends. No quantitative offline model results are claimed because reproducible evaluation metrics were not supplied.

Index Terms— e-commerce analytics, customer segmentation, RFM, association rule mining, collaborative filtering, Olist, PostgreSQL.

I. INTRODUCTION

Online retail systems collect order, item, payment, product, customer, seller, and review records. An analytical application must join these sources consistently and translate them into outputs users can inspect. This project implements that workflow as a locally operated web application over PostgreSQL using the public Olist dataset.

The engineering question is how a multi-table transaction dataset can support one interface for descriptive analytics, customer analysis, product affinity, forecasting, and personalized suggestions. The system uses a React and TypeScript frontend, a Python FastAPI service, and PostgreSQL. Analytical jobs execute through the API and persist results.

The contributions are: 1) an end-to-end application integrating data access, analytics, and visualization; 2) compatibility views that map Olist data to the existing analytics schema while preserving source tables; and 3) a purchase-history recommender with transparent peer similarity and a popularity fallback. This is a system implementation, not a novel-algorithm or commercial-impact claim.

II. DATASET AND DATA MODEL

The Brazilian E-Commerce Public Dataset by Olist contains approximately 100,000 orders from 2016-2018 and includes order, product, customer, payment, seller, review, and location information [1]. It is anonymized and distributed as nine CSV files. The tables are customers, geolocation, order items, payments, reviews, orders, products, sellers, and product category translations. Orders join to customer records and items; items join to products and sellers. Olist order-specific customer IDs can be linked across repeat purchases using customer_unique_id [1].

Table group	One row represents
Customers / sellers	Buyer or seller and location
Orders	Order status and lifecycle timestamps
Order items	One product line, seller, price, freight
Payments	Payment method, installments, amount
Reviews	Review score, comment, timestamps
Products	Catalog category and measurements
Geolocation	ZIP-prefix coordinate record
Translation	Portuguese and English category

A. Compatibility and assumptions

A SQL script creates read-only views customers, products, orders, and order_items, plus a stable integer customer mapping and persisted-result tables. Source Olist tables remain intact. The compatibility item view assigns quantity one because the source has no quantity column; item price is used as revenue and freight is excluded. The shared order view includes delivered, canceled, and unavailable statuses and flags the latter two as cancellations. Recommendations use delivered orders only. Values remain in Brazilian reais (BRL).

III. SYSTEM ARCHITECTURE

The React and TypeScript frontend sends HTTP/JSON requests to a Python FastAPI backend. SQLAlchemy connects the API to PostgreSQL. Each analysis request creates a run record; a background task updates its status and saves results. Pages retrieve persisted results rather than fitting models on every view request.

Layer	Implementation
Presentation	React, TypeScript, Vite, charts
API	Python, FastAPI, validation
Analytics	Pandas, NumPy, scikit-learn, mlxtend
Storage	PostgreSQL source tables, views, runs

The /api/health endpoint checks database access. /api/data-source reports detected schema, currency, data presence, and analytics readiness. Dashboard, customer, RFM, clustering, basket, forecast, recommendation, import, and run-history endpoints supply the UI. Jobs currently run in the API process, not a distributed queue.

IV. ANALYTICAL METHODS

A. Dashboard and RFM

The dashboard summarizes revenue, orders, customers, average order value, repeat purchases, cancellations, trends, products, and geography. RFM recency is days from the last purchase to the day after the dataset's last date; frequency is distinct orders; monetary value is non-canceled item revenue. Rank-based quintiles assign scores 1–5, with recency reversed so recent buyers score higher. Rule-based labels include Champions, Loyal Customers, At Risk, and Hibernating.

B. Customer segmentation

The pipeline applies log(1+x) to recency, frequency, and monetary features, then standardizes them. It evaluates k from 2 to 8, bounded by customer count, using silhouette score and selects the best unless k is fixed. K-Means uses a fixed seed; cluster profiles and a two-dimensional PCA projection are saved [2]. Clusters are descriptive, not ground-truth classes.

C. Market-basket analysis

The system forms invoice-by-product binary baskets, considers up to 150 frequent products by default, removes single-item baskets, and mines rules with Apriori. For $X \rightarrow Y$, support is the share of transactions containing $X \cup Y$, confidence estimates $P(Y|X)$, and lift compares confidence with the marginal frequency of Y. Itemset length is bounded to three by default [3].

D. Revenue forecasting

Daily revenue fills missing dates with zero. A Gradient Boosting Regressor uses weekday, day of month, month, lags of 28, 35, and 42 days, and a shifted seven-day rolling mean. The final 28 days form a chronological test window. Test MAE and RMSE are compared with a seasonal-naive baseline repeating the last observed weekday values. The model is refitted on all history for a forecast horizon up to 28 days [4]. At least 120 days plus the test window are required. No measured metrics are reported here.

V. PERSONALIZED RECOMMENDATIONS

The recommender uses delivered purchase histories as sets of distinct product IDs. For target customer u and another customer v, Jaccard similarity is:

J(u,v) = shared products in the two histories / all distinct products across both histories

Up to 100 overlapping customers are retained. An unseen product receives a score equal to the sum of Jaccard similarities for neighbors who bought it.

Candidates rank by score and supporting-neighbor count. If peer candidates are absent, popular delivered items not in the target history are used.

The interface shows customer history, product IDs, translated categories, and the number of similar customers supporting a recommendation. Olist lacks readable product titles. The method uses purchases only, with no impressions, browsing, cart events, or explicit ratings. It is an interpretable baseline, not a production recommender.

VI. IMPLEMENTATION AND WORKFLOW

Operators import the nine CSVs into PostgreSQL in foreign-key order and run compatibility SQL after loading. The backend connection is configured in backend/.env. Pages include Dashboard, Customers & RFM, Segmentation, Products & Baskets, Recommendations, Forecasting, Data Import & Quality, and Run History. Users inspect data readiness, explore metrics, trigger analyses, and review persisted results. Workbook import is UCI-specific; Olist CSVs are loaded through pgAdmin. The project owner reports the Olist application and recommendations are working.

VII. EVALUATION STATUS AND DISCUSSION

This paper documents implementation and methodology. It does not claim measured accuracy, relevance, runtime, usability, or business impact. No held-out recommendation test, forecast metric output, or reproducible benchmark was provided. A populated interface demonstrates an execution path, not model quality.

A next evaluation should train on earlier delivered orders, hold out later interactions, and compare the recommender with popularity using Precision@K, Recall@K, catalog coverage, and customer-activity groups. Forecast MAE and RMSE should be reported against the seasonal baseline on fixed dates, in BRL. Clustering should include cluster sizes, profiles, and silhouette score. Actual database counts and date range should be captured from the loaded database.

Olist is historical and Brazil-specific, so it cannot substantiate current Indian trends. The compatibility layer simplifies quantity and excludes freight from item revenue. Sparse histories cause cold-start and limited-overlap cases. Geolocation, payments, and reviews are imported but are not central to current displayed analytics. Background jobs in the API process suit a local demonstration but are not resilient to process restarts.

VIII. CONCLUSION

This platform connects Olist relational data with customer analysis, product affinity, revenue forecasting, and personalized suggestions. Its engineering contribution integrates database setup, schema adaptation, persisted runs, API routes, and user-facing pages. The recommender uses Jaccard similarity on delivered histories and a popularity fallback. Quantitative evaluation, additional Olist signals, and a temporally newer dataset are important next steps.

REFERENCES

[1] Olist, “Brazilian E-Commerce Public Dataset by Olist,” Kaggle, 2018. [Online]. Available: kaggle.com/datasets/olistbr/brazilian-ecommerce

[2] J. MacQueen, “Some methods for classification and analysis of multivariate observations,” Proc. 5th Berkeley Symp. Math. Statist. Probability, vol. 1, 1967, pp. 281–297.

[3] R. Agrawal and R. Srikant, “Fast algorithms for mining association rules in large databases,” Proc. 20th Int. Conf. Very Large Data Bases, 1994, pp. 487–499.

[4] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, 2001, doi: 10.1214/aos/1013203451.